arXiv:2411.06208v3 [cs.CL] 17 Jul 2025

IOPO: Empowering LLMs with Complex Instruction Following via Input-Output Preference Optimization

Xinghua Zhang, Haiyang Yu, Cheng Fu, Fei Huang, Yongbin Li*
Tongyi Lab, Alibaba Group
{zhangxinghua.zxh, yifei.yhy, fucheng.fuc, f.huang, shuide.lyb}@alibaba-inc.com

Abstract

In the realm of large language models (LLMs), the ability of models to accurately follow instructions is paramount as more agents and applications leverage LLMs for construction, where the complexity of instructions are rapidly increasing. However, on the one hand, there is only a certain amount of complex instruction evaluation data; on the other hand, there are no dedicated algorithms to improve the ability to follow complex instructions. To this end, this paper introduces **TRACE**, a benchmark for improving and evaluating the complex instruction-following ability, which consists of 120K training data and 1K evaluation data. Furthermore, we propose **IOPO** (Input-Output Preference Optimization) alignment method which takes both input and output preference pairs into consideration, where LLMs not only rapidly align with response preferences but also meticulously explore the instruction preferences. Extensive experiments on both in-domain and out-of-domain datasets confirm the effectiveness of IOPO, showing 8.15%, 2.18% improvements on in-domain data and 5.91%, 2.83% on out-of-domain data compared to SFT and DPO respectively. Our code and dataset are released at https://github.com/AlibabaResearch/DAMO-ConvAI/tree/main/IOPO.

1 Introduction

The rapid development of LLMs has facilitated human-machine interaction, with instructions serving as the medium (Gao et al., 2024; Kim et al., 2024; Zhang et al., 2024c; Wang et al., 2024b; Zhang et al., 2024b). As human needs evolve, there is an increasing expectation for models to handle more intricate tasks through complex instructions (Ge et al., 2023; Yang et al., 2024b; Wang et al., 2024a). Consequently, the instruction-following ability, especially complex instructions, is garnering significant attention (Zhou et al., 2023; Xu et al., 2024; Li et al., 2024; Zhang et al., 2024a).

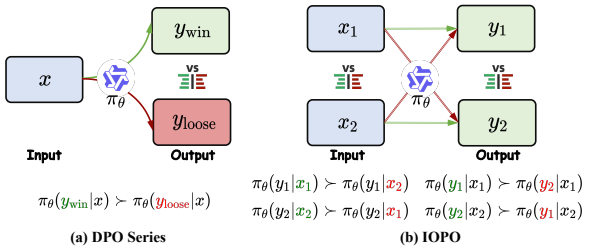

Figure 1: Alignment Paradigms (a) Existing DPO Series vs. (b) Proposed IOPO. The green arrow indicates that y matches x while the red one indicates a mismatch.

To evaluate the instruction-following abilities of LLMs, several benchmarks (Zhou et al., 2023; Qin et al., 2024; Li et al., 2024) have been proposed, which are designed to systematically assess how well these models can understand and execute instructions. IFEval (Zhou et al., 2023) focuses on verifiable instructions which are amenable to objective verification of compliance. Qin et al. (2024) introduces INFOBENCH which contains 2,250 decomposed questions to assess the instruction following. Recently, the ability to follow complex instructions with multiple constraints is gaining increasing attention (He et al., 2024b; Jiang et al., 2024; Wen et al., 2024; He et al., 2024a) as LLMs are deployed in sophisticated real-world applications. Zhang et al. (2024a) proposes a constraint-following benchmark CFBench with 1,000 multi-constraint samples. However, most of benchmarks lay emphasis on evaluating LLMs' ability to follow complex instructions, lack of algorithms tailored for enhancing the corresponding ability.

From RLHF (Reinforcement Learning from Human Feedback) (Ouyang et al., 2022; Bai et al., 2022a) to the following-up researches such as DPO (Direct Preference Optimization) (Rafailov et al., 2023), alignment algorithms which align LLMs with human preferences, have demonstrated their

*Corresponding author.

effectiveness in improving the LLMs' capabilities to follow instructions. Nevertheless, these methods directly explore different responses y (y_{win}, y_{loose}) based on the same instruction x, as shown in Figure 1 (a). In the complex instruction scenario which contains multiple constraints, it is challenging to efficiently perceive the fine-grained constraints in x solely by modeling different y.

To bridge this gap, this paper first introduces **TRACE** benchmark to improve the ability of LLMs to **t**rack complex fine-g**r**ained constr**a**int instru**c**tions and make them more ob**e**dient. TRACE is automatically constructed based on the manually sorted taxonomy of complex instructions with 26 constraint dimensions within 5 constraint types. We develop an automated data construction workflow that extends from open-source simple instructions to multi-constrained complex ones. In the end, we accumulate 120K complex instructions for model training and 1K human-verified data for evaluation. To enhance the ability of LLMs to follow complex instructions, this paper further proposes *Input-Output Preference Optimization* (**IOPO**) method. IOPO not only takes the instruction x as input to directly learn the response y preference, but also gradually delves deeper into instructions x based on the same response y, to promote effective perception of fine-grained constraints, as shown in Figure 1 (b). The major contributions of this paper are summarized as follows:

- We introduce a benchmark **TRACE** for complex instruction following, which includes both an evaluation set and a training set, and an automated data construction workflow, further enriching the research community.
- Different from previous alignment paradigm, we propose **IOPO** alignment method which deeply explores the complex instructions x (*Input*), not just directly learning response preference y (*Output*).
- Extensive experiments on both in-domain and out-of-domain evaluations have confirmed the consistent improvements, with an average increase of 7.03% and 2.51%, compared to SFT and DPO, respectively.

2 Related Work

2.1 Instruction Following

Instruction following is the most fundamental and crucial ability for large language models (LLMs), which enables them to understand and execute user instructions accurately, making them more effective in a wide range of applications. In fact, earlier studies have explored the extent to which models follow language instructions (Ye and Ren, 2021; Mishra et al., 2022; Hase and Bansal, 2022). It is effective to fine-tune LLMs on these annotated instruction data for improving the ability to follow natural language instructions. The instruction-following ability enhances adaptability to unseen tasks, which has become an efficient learning paradigm for novel task demands (Lou et al., 2023).

As human demands grow higher, the instructions given to LLMs are also becoming increasingly complex. Recent studies are beginning to focus on the complex instruction-following ability of LLMs, where more complex or constrained instructions have been proven effective in enhancing LLMs' abilities to follow instructions (Mukherjee et al., 2023; Xu et al., 2024; Luo et al., 2024). Constrained instructions, as a type of complex instruction, are also gradually receiving attention from the research community. Increasing the complexity of constraints within the instruction (e.g., raising the number of constraints) can further improve the ability to follow complex instructions (Sun et al., 2024; Dong et al., 2024; He et al., 2024a). Sun et al. (2024) introduces a instruction tuning dataset Conifer and proposes a progressive learning scheme to enhance the ability of LLMs to follow multi-level instructions with complex constraints. He et al. (2024a) first finds that multi-constraint instructions can enhance LLMs' understanding of complex instructions, and then introduces a discrimination-based method for data acquisition, and finally proposes a contrastive method with reinforcement learning fine-tuning (RLFT) for data utilization. In addition, some work focuses on evaluating the multi-constraint instruction-following capabilities of LLMs (Zhou et al., 2023; Qin et al., 2024; Jiang et al., 2024). Zhang et al. (2024a) introduces CFBench, a benchmark which encompasses instructions with multiple constraints, and proposes a multi-dimensional evaluation framework to comprehensively assess model capabilities.

2.2 LLM Alignment

LLM alignment aims to enhance LLMs by aligning them with human preference. Recent research has conducted extensive explorations for LLM alignment. From RLHF/PPO (Ouyang et al., 2022; Bai et al., 2022a) to DPO (Rafailov et al., 2023) and

beyond (Bai et al., 2022b; Song et al., 2024; Meng et al., 2024), the evolution of xPO-series alignment algorithms has seen significant advancements. These methods have been pivotal in improving the alignment between LLMs and human values, ensuring that the outputs of these models are not only effective but also follow human preference.

RLHF involves training models using human-provided rewards or reward models to improve decision-making, optimizing policies through iterative feedback loops for more aligned outcomes. DPO directly optimizes the model's output to match preferred response as indicated by human feedback, which simplifies the alignment process by focusing on direct comparisons between preferred and dispreferred outputs, allowing the model to learn human preference without needing an explicitly defined reward model. SimPO (Meng et al., 2024) proposes a method for preference optimization that eliminates the need for the reference model π_{ref}, which is memory efficient and simple. Instead of using pairwise data, PRO (Song et al., 2024) utilizes the preference ranking of any length listwise preference dataset. ORPO (Hong et al., 2024) introduces the reference model-free preference optimization algorithm.

To further enrich the community of multi-constraint instruction following ability, we construct TRACE benchmark which contains instructions with multiple constraints, more fine-grained constraint types and a wider range of constraint quantities. In addition, we propose the tailor-designed alignment algorithm IOPO for fine-grained multi-constraint alignment, which is different from previous methods (e.g., DPO) only focusing on the output preference.

3 Preliminary

Existing alignment methods have evolved from RLHF (Ouyang et al., 2022; Bai et al., 2022a), this section provides a brief introduction to RLHF which mainly consists of three stages:

1) **SFT**: The generic pre-trained LM is fine-tuned with maximum likelihood supervised loss on downstream task data, and then we can get the SFT model π_{SFT}.

2) **Reward Model**: The model π_{SFT} is utilized with prompt x to generate two different responses y_1, y_2. The pair of responses is labeled as "preferred" and "dispreferred" by human labelers, i.e., $y_1 \succ y_2 \mid x$. The reward model r_ϕ is trained with the following negative log-likelihood loss:

$$\mathcal{L}_R = -\mathbb{E}_{(x,y_1,y_2)\sim\mathcal{D}}[\log \sigma(r_\phi(x, y_1) - r_\phi(x, y_2))] \tag{1}$$

3) **RL**: This stage uses the learned reward model r_ϕ to provide feedback to the language model policy, the optimization objective is as follows:

$$\max_{\pi_\theta} \mathbb{E}_{x\sim D, y\sim\pi_\theta(y|x)}[r_\phi(x, y)] - \beta\mathbb{D}_{\text{KL}}[\pi_\theta(y|x)||\pi_{\text{ref}}(y|x)] \tag{2}$$

where LM policy π_θ, base reference policy π_{ref} are both initialized with π_{SFT}, β controls the deviation of π_θ from the base policy π_{ref}.

4 TRACE Benchmark

This section describes the construction pipeline of TRACE, its statistics, and evaluation protocol.

4.1 Construction Pipeline

The overall construction process as shown in Figure 2 includes several key stages: 1) *Taxonomy of Constraint*, 2) *Constraint Expansion*, 3) *Instruction Structuring*, 4) *Quality Control*, and 5) *Response Generation & Evaluation*.

Taxonomy of Constraint. A comprehensive constraint type system is developed through inference by LLM from a large volume of open-source simple instructions, and further refined by human experts, into 5 constraint types (Jiang et al., 2024) and 26 constraint dimensions. The detailed description of constraints is shown in Appendix A.

Constraint Expansion. This step aims to expand simple instructions into more complex ones that incorporate multiple constraints based on the taxonomy of constraint by prompting LLM.

Instruction Structuring. To better distinguish different segments of the instruction, this step structures the flat instruction text expanded from the last step into *Task Description*, *Constraints*, and *Input* part by prompting LLM.

Quality Control. To ensure the validity of the instructions, this step conducts quality control of the expanded instructions by prompting LLM, addressing some forms of invalidity such as redundancy between the description and constraints, incompleteness between the description and input.

Response Generation & Evaluation. First, we prompt LLM with the instruction x to generate the corresponding response y. To confirm its quality, we then prompt LLM to rate how well the response y comply with constraints in the instruction

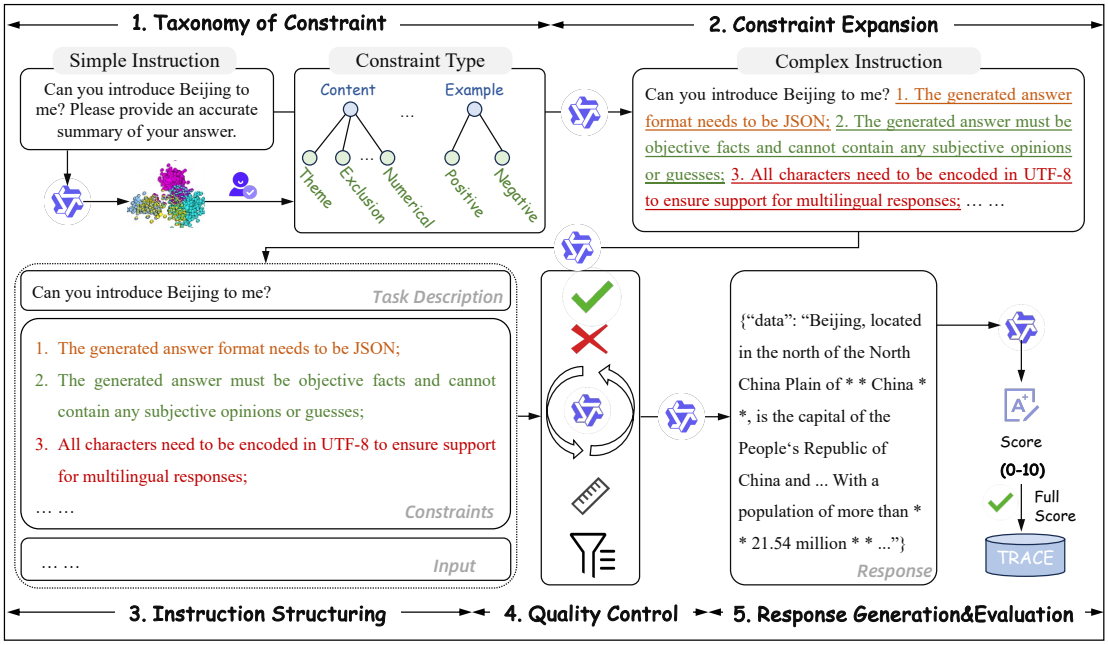

Figure 2: Construction Pipeline of TRACE, which consists of five key stages: 1) *Taxonomy of Constraint*, 2) *Constraint Expansion*, 3) *Instruction Structuring*, 4) *Quality Control*, and 5) *Response Generation & Evaluation*.

	#N	Min.	Max.	Avg.
#Training	119,345	1	15	4.36
#Evaluation	1,042	1	15	4.89

Table 1: The statistics of TRACE benchmark. #N is the number of instructions; Min., Max., and Avg. mean the minimum, maximum, and average number of constraints per instruction.

x. Finally, data that fully follows all the constraints outlined in the instruction would receive a perfect score of 10, and is selected to form the supervised fine-tuning (SFT) instruction dataset.

The corresponding prompts for constraint expansion, instruction structuring, quality control, response generation and evaluation are shown in Appendix B.

4.2 Dataset Statistics

As shown in Table 1, TRACE consists of 119,345 instructions for model training, and 1,042 instructions for evaluation, where the minimum and maximum number of constraints per instruction are 1 and 15, with average numbers of 4.36 and 4.89, respectively. Table 2 gives the constraint number distributions over training and evaluation set in TRACE. For example, when $\mathcal{C} = 6$, the corresponding column indicates that there are 13,858 instructions with 6 constraints in the training set, and 100 instructions with 6 constraints in the evaluation set. In Figure 3, we depict the constraint type distribution over the evaluation set. The inner circle represents the distribution of five major types (*Content Constraint*, *Situation Constraint*, *Style Constraint*, *Format Constraint*, and *Example Constraint*), and the corresponding outer one represents the distribution of concrete constraint dimensions.

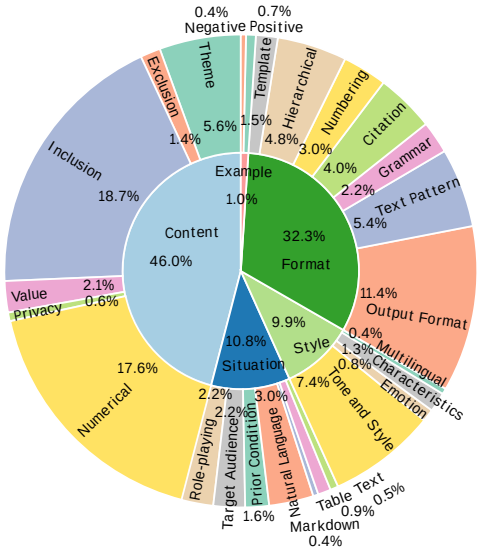

Figure 3: Constraint type distribution over evaluation set in TRACE.

4.3 Evaluation Protocol

Following previous work (Qin et al., 2024; Zhang et al., 2024a), we use GPT-4o as the evaluator to as-

$\mathcal{C}=$	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
#Training	991	8,003	26,421	34,155	26,327	13,858	5,882	2,185	999	464	8	20	20	8	4
#Evaluation	200	100	100	100	100	100	100	100	100	10	10	10	4	4	4

Table 2: Constraint number ($\mathcal{C}$) distributions over training and evaluation set in TRACE. $\mathcal{C}=i$ represents the number of instructions with i constraints.

sess the generated response based on the complex instruction. Concretely, we prompt the LLM evaluator to evaluate each constraint mentioned in the complex instruction on a scale of 0–10, assessing the degree to which the response follows each constraint. A higher score indicates stronger adherence to the specified constraint. The overall instruction following score IF on the evaluation set with n complex instructions are calculated as follows:

$$\text{IF} = \frac{1}{n}\sum_{i}^{n}\mathbb{I}_{=10}\sum_{j=1}^{m_i}\frac{\mathcal{S}_{i,j}}{m_i} \tag{3}$$

where $\mathcal{S}_{i,j} \in [0, 10]$ is the score indicating the degree to which the j-th constraint in the i-th instruction is adhered to. m_i is the number of constraints in the i-th instruction. $\mathbb{I}_{=10}$ is 1 when $\sum_{j=1}^{m_i}\frac{\mathcal{S}_{i,j}}{m_i} = 10$, otherwise is 0. That is, a response is considered correct only when all constraints in the complex instruction are fully followed.

4.4 Evaluation Set Quality

To generate the high-quality evaluation set, we further introduce a rigorous post-inspection process after the construction pipeline (Sec. 4.1). First, based on LLM-as-Judge (Wang et al., 2024c; Zhang et al., 2023; Zeng et al., 2024; Xia et al., 2024), we use the powerful LLM GPT-4o to check the following items for each instruction in the evaluation set: 1) Is the description empty? 2) Is there redundancy between the constraints and description? 3) Does the input match the description? If any of the aforementioned issues arise, we prompt the GPT-4o to make corrections for accelerating the subsequent manual verification. Second, the professional data annotation team makes the manual annotation process involving multiple steps such as annotator training, small-scale trial annotation, selection of official annotators, and formal annotation. Finally, we randomly select 100 instructions for quality evaluation, which are then inspected by three labeling specialists based on the above check items and the overall validity. The agreement rate among three annotators on the sampled evaluation set is 95%.

5 Input-Output Preference Optimization

Both RLHF (Ouyang et al., 2022; Bai et al., 2022a) and its variants, such as DPO (Rafailov et al., 2023), directly learn the response y preference (*Output*) given the same instruction x (*Input*). However, complex instructions consist of multiple fine-grained constraints, direct preference learning for the output y struggles to perceive fine-grained constraints in the input x. To enhance the model's perception of fine-grained instruction, we further introduce the input preference learning which reflects on the constraints in the instruction x based on the response y. By performing preference learning of both input and output, **i**nput-**o**utput **p**reference **o**ptimization (IOPO) not only rapidly fits the better output but also meticulously considers the fine-grained information of the input.

Concretely, we construct a pair of instructions <x_1, x_2> whose responses are respectively <y_1, y_2>, where x_2 has subtle differences from x_1 in some constraints, and these differences would result in substantially divergent responses. And then, we can get four input-output pairs <x_1, y_1>, <x_1, y_2>, <x_2, y_1>, and <x_2, y_2>, which can form a preference group pair $\mathcal{G}_1 \succ \mathcal{G}_2$ ($\mathcal{G}_1$ = {<x_1, y_1>, <x_2, y_2>}, $\mathcal{G}_2$ = {<x_1, y_2>, <x_2, y_1>}). The detailed data construction process is described in Appendix D. The first group is the matched input-output pair while the second one is mismatched. As derived in Rafailov et al. (2023), the reward function $r(x, y)$ can be represented by the policy model π_r as follows:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \tag{4}$$

where $Z(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\left(\frac{1}{\beta} r(x, y)\right)$.

The Bradley–Terry model (Bradley and Terry, 1952) is a probability model for the outcome of pairwise comparisons between items, groups, or objects. Given a pair of items i and j, it estimates the probability that the pairwise comparison $i \succ j$ turns out true (Hunter, 2004), as

$$p(i \succ j) = p_i/(p_i + p_j) \tag{5}$$

where p_i is a positive real-valued score assigned to individual i, and the comparison $i \succ j$ can mean "i is preferred to j". Similarly, given a pair of groups $\mathcal{G}_1$ and $\mathcal{G}_2$, we can define $p_1 = e^{r(x_1,y_1)+r(x_2,y_2)}$ for $\mathcal{G}_1$, and $p_2 = e^{r(x_1,y_2)+r(x_2,y_1)}$ for $\mathcal{G}_2$, as

$$\begin{aligned} p(\mathcal{G}_1 \succ \mathcal{G}_2) &= \frac{e^{r_{\mathcal{G}_1}}}{e^{r_{\mathcal{G}_1}} + e^{r_{\mathcal{G}_2}}} \\ r_{\mathcal{G}_1} &= r(x_1, y_1) + r(x_2, y_2) \\ r_{\mathcal{G}_2} &= r(x_1, y_2) + r(x_2, y_1) \end{aligned} \tag{6}$$

Next, combining Eq. 6 and Eq. 4, we can further derive as follows (details in Appendix G):

$$p(\mathcal{G}_1 \succ \mathcal{G}_2) = \sigma\left(\frac{1}{2}(\Pi_1 + \Pi_2)\right) \tag{7}$$

$$\begin{aligned} \Pi_1 =& 2\beta \log \frac{\pi_r(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi_r(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} \\ & - \beta \log \frac{\pi_r(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)} \\ \Pi_2 =& 2\beta \log \frac{\pi_r(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} - \beta \log \frac{\pi_r(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)} \\ & - \beta \log \frac{\pi_r(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} \end{aligned} \tag{8}$$

where σ is the sigmoid function. Therefore, the optimization objective of IOPO is to maximize $p(\mathcal{G}_1 \succ \mathcal{G}_2)$. Motivated by Rafailov et al. (2023), we can formulate a maximum likelihood loss for a parametrized policy model π_θ as follows:

$$\begin{aligned} \mathcal{L}_{\text{IOPO}}(\pi_\theta) &= -\mathbb{E}_{i\sim D}\left\{\log\left[\sigma\left(\frac{\Pi_1(\pi_\theta) + \Pi_2(\pi_\theta)}{2}\right)\right]\right\} \\ i &= < x_1, y_1, x_2, y_2 > \end{aligned} \tag{9}$$

$$\begin{aligned} \Pi_1(\pi_\theta) =& \Bigg(\underbrace{\beta \log \frac{\pi_\theta(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi_\theta(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)}}_{\texttt{Output}}\Bigg) \\ &+ \Bigg(\underbrace{\beta \log \frac{\pi_\theta(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi_\theta(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)}}_{\texttt{Input}}\Bigg) \\ \Pi_2(\pi_\theta) =& \Bigg(\underbrace{\beta \log \frac{\pi_\theta(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} - \beta \log \frac{\pi_\theta(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)}}_{\texttt{Output}}\Bigg) \\ &+ \Bigg(\underbrace{\beta \log \frac{\pi_\theta(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} - \beta \log \frac{\pi_\theta(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)}}_{\texttt{Input}}\Bigg) \end{aligned} \tag{10}$$

where we mark the preference modeling for `Output` and `Input` in $\Pi_1(\pi_\theta)$, $\Pi_2(\pi_\theta)$.

6 Experiments

6.1 Experimental Settings

Evaluation Datasets. We conduct experiments on four instruction-following datasets: TRACE, IFEval (Zhou et al., 2023), CFBench (Zhang et al., 2024a), and COMPLEXBENCH (Wen et al., 2024). TRACE evaluation set is introduced in this paper, which has 1,042 instructions, and an average of 4.89 constraints per instruction, with a maximum of 15 constraints. IFEval consists of 541 prompts, with each prompt containing one or multiple verifiable instructions. CFBench contains 1,000 samples that cover more than 200 real-life scenarios and over 50 NLP tasks, with each sample including multiple constraints. COMPLEXBENCH creates 1,150 samples based on 4 constraint types, 19 constraint dimensions, and 4 composition types. It is worth noting that TRACE is the in-domain evaluation set, IFEval, CFBench and COMPLEXBENCH are the out-of-domain ones.

Implementation Details. (1) **TRACE Benchmark**: we choose Qwen2-72B-Instruct (Yang et al., 2024a)[1] for benchmark construction. (2) **IOPO Alignment**: we choose Qwen2-7B-Instruct[2], and LLaMA3.1-8B-Instruct[3] as the LLM backbone. All models, except for the base models (i.e. Qwen2-7B-Instruct and Llama-3.1-8B-Instruct), are trained on TRACE's training set or its variants, tested on all evaluation datasets (*Train Once, Test Anywhere*). The learning rate is 1e-4 for supervised fine-tuning (SFT), and 5e-6 for DPO and IOPO. The maximum length and epoch are set to 6,000 and 3 respectively. β is set to 0.1. We implement our code based on LLaMA-Factory (Zheng et al., 2024), perform parallel training on 4 $\times$ 8-GPU machines, with a micro batch size of 1 per GPU. The DPO training data construction is shown in Appendix C.

Evaluation Metrics. For TRACE, we use GPT-4o to evaluate if all constraints in the instruction have been followed (IF-S for single-constraint instructions, and IF-M for multi-constraint instructions), as described in Sec. 4.3. For IFEval, we use prompt-level strict and loose accuracy defined in Zhou et al. (2023), abbr. S-Acc and L-Acc respectively. CFBench (Zhang et al., 2024a) introduces three evaluation metrics with GPT-4o as the evaluation model: constraint satisfaction rate (CSR), instruction satisfaction rate (ISR), and priority satisfaction rate (PSR). COMPLEXBENCH (Wen et al., 2024) calculates the decomposed requirements following ratio (DRFR).

[1]https://modelscope.cn/models/Qwen/Qwen2-72B-Instruct

[2]https://modelscope.cn/models/Qwen/Qwen2-7B-Instruct

[3]https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

Model	Method	TRACE		IFEval		CFBench			COMPLEXBENCH
		IF-S	IF-M	S-Acc	L-Acc	CSR	ISR	PSR	DRFR
Qwen2-7B	Instruct	72.5	54.5	51.6	56.4	75.8	39.1	50.2	68.1
	SFT	76.0	56.1	52.3	54.2	77.8	40.4	52.9	68.2
	PPO	77.0	57.7	51.4	53.8	76.2	38.8	50.6	68.6
	ORPO	77.9	61.7	53.1	56.9	79.7	45.9	57.0	69.1
	SimPO	78.3	63.6	52.2	57.6	78.4	45.0	57.6	67.8
	DPO	79.0	67.2	52.7	58.2	80.0	45.1	57.9	70.9
	IOPO (Ours)$_{Improv.}$	82.0$_{\uparrow 3.0}$	68.9$_{\uparrow 1.7}$	59.9$_{\uparrow 7.2}$	63.6$_{\uparrow 5.4}$	80.7$_{\uparrow 0.7}$	47.0$_{\uparrow 1.9}$	58.7$_{\uparrow 0.8}$	72.6$_{\uparrow 1.7}$
Llama3.1-8B	Instruct	67.5	52.9	74.3	78.6	71.4	35.7	46.9	63.2
	SFT	75.5	62.9	71.0	74.1	78.4	43.2	54.7	68.2
	PPO	75.0	57.3	69.9	72.3	75.9	40.9	50.7	68.1
	ORPO	77.0	63.1	72.3	77.3	79.4	46.6	57.2	69.6
	SimPO	76.3	64.5	71.2	76.6	80.6	47.8	58.7	70.2
	DPO	79.0	69.2	71.5	76.5	80.8	48.1	59.8	70.8
	IOPO (Ours)$_{Improv.}$	81.5$_{\uparrow 2.5}$	70.7$_{\uparrow 1.5}$	78.2$_{\uparrow 6.7}$	81.0$_{\uparrow 4.5}$	81.8$_{\uparrow 1.0}$	49.9$_{\uparrow 1.8}$	61.1$_{\uparrow 1.3}$	71.8$_{\uparrow 1.0}$

Table 3: Main results on in-domain TRACE, and out-of-domain IFEval, CFBench, and COMPLEXBENCH. *Improv.* indicates the absolute improvement compared to DPO.

Model	Method	TRACE		IFEval		CFBench			COMPLEXBENCH
		IF-S	IF-M	S-Acc	L-Acc	CSR	ISR	PSR	DRFR
Qwen2-7B	IOPO	82.0	68.9	59.9	63.6	80.7	47.0	58.7	72.6
	w/o Output Preference	81.0	66.7	55.1	60.5	79.4	46.6	56.3	71.0
	w/o Input Preference	80.9	67.1	56.7	61.9	79.7	46.8	57.0	71.3
Llama3.1-8B	IOPO	81.5	70.7	78.2	81.0	81.8	49.9	61.1	71.8
	w/o Output Preference	81.5	69.6	77.3	80.6	80.6	48.6	58.4	69.2
	w/o Input Preference	79.0	69.0	77.9	80.2	80.9	48.3	59.4	70.1

Table 4: Ablation studies on TRACE, IFEval, CFBench, and COMPLEXBENCH.

6.2 Experimental Results

Main Results. As shown in Table 3, we give the main results under different benchmarks, including in-domain TRACE, out-of-domain IFEval, CFBench and COMPLEXBENCH. The experiments are conducted under two different base models, Qwen2-7B, and Llama3.1-8B, where Instruct means directly using Qwen2-7B-Instruct or Llama3.1-8B-Instruct for inference, SFT represents the model is trained on TRACE training set, and PPO, DPO, IOPO are respectively trained on preference data derived from TRACE training set.

For in-domain evaluation on TRACE set, we can see 3.0%, 1.7% improvements of IOPO on single- and multi-constraint instructions with Qwen2-7B as the base model compared to DPO, and 2.5%, 1.5% improvements with Llama3.1-8B as the base model. For out-of-domain evaluation on IFEval, CFBench and COMPLEXBENCH, IOPO achieves an average increase of 2.95%, and 2.72% in comparison with DPO based on Qwen2-7B and Llama3.1-8B respectively. The significant advantages of both in-domain and out-of-domain evaluations confirm the effectiveness of input-output preference optimization, which intensively considers the constraint differences between instructions, enhancing the model's perception of constraints. It is worth noting that IOPO has a larger performance gap with SFT especially on IFEval, compared to DPO and SFT, which confirms the generalization of IOPO and the necessity of further modeling the input preferences.

Ablation Studies. To further confirm the effectiveness of input and output preference, we conduct

Method	SFT	DPO	IOPO
#Memory	1×	2×	4×
#Training Time	14.54 h	26.30 h	34.27 h
#Inference Speed	1×	1×	1×

Table 5: Analysis on the consumed GPU memory, training time, and inference speed under the same batch size.

ablation studies on TRACE, IFEval, CFBench, and COMPLEXBENCH as shown in Table 4, where "w/o Output Pref" [4] means we only consider the modeling of input preference with the same training data, "w/o Input Pref" [5] means we only consider the modeling of output preference. We see that *output preference* contributes to 2.1%, and 1.28% increases with Qwen2-7B and Llama3.1-8B respectively, *input preference* separately brings 1.5% and 1.4% performance gains, which confirms the effectiveness of both input and output preference modeling. Besides the paradigm for modeling output preference in existing alignment methods, it's established that modeling input preference is crucial for deeply considering constraints within the instruction.

Complexity Analysis. We conduct the analyses of complexity in Table 5, where all methods are conducted under the same experimental settings, such as the batch size and GPU. (1) For #Memory, DPO and IOPO are approximately twice and four times that of SFT respectively, because DPO needs a pair of responses to calculate the corresponding loss (<x, y_1>, <x, y_2>), and IOPO needs to compute four groups of input-output pairs (<x_1, y_1>, <x_2, y_2>, <x_1, y_2>, <x_2, y_1>) in its loss. (2) For #Training Time, DPO and IOPO require the computation of more tokens compared to SFT under the same batch size, leading to longer training time. (3) For #Inference Speed, SFT, DPO, and IOPO are all the same base model optimized for inference, resulting the same inference speed. The training efficiency and GPU memory usage of IOPO are not the best among compared baselines, but their efficiencies are still of the same order of magnitude, which are reasonable and acceptable comprehensively considering its performance advantage.

[4]

$$\mathcal{L}_{\text{IOPO}^*}(\pi_\theta) = -\mathbb{E}_{i\sim D}\log[\sigma(\Pi^*(\pi_\theta))]$$

$$i =< x_1, y_1, x_2, y_2 >$$

$$\Pi^*(\pi_\theta) = \beta \log \frac{\pi_\theta(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi_\theta(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)}$$

where the preference pair data $y_2|x_2$, $y_2|x_1$ are also used for training.

[5]Different from "w/o Output Pref",

$$\Pi^*(\pi_\theta) = \beta \log \frac{\pi_\theta(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi_\theta(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)}$$

where the preference pair data $y_2|x_2$, $y_1|x_2$ are also used for training.

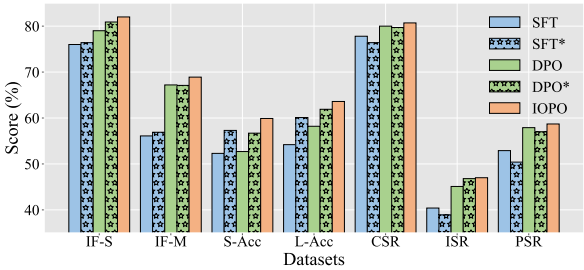

Figure 4: Performance comparisons under the same quantity of tokens with **Qwen2-7B** as the base model.

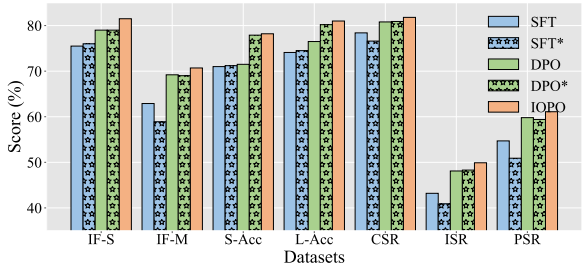

Figure 5: Performance comparisons under the same quantity of tokens with **Llama3.1-8B** as the base model.

The Impact of Token Quantity. To address concerns regarding the IOPO training tokens, we conduct the analyses on the impact of token quantity and report the results in Figure 4, and Figure 5. For IOPO, there exist two instructions along with their corresponding/right responses ({<x_1, y_1>, <x_2, y_2>}). To ensure that DPO and IOPO consume the same number of tokens, we construct two pairs of output preferences based on IOPO's instructions x_1 ($y_{\text{win}} = y_1$, $y_{\text{lose}} = y_2$), and x_2 ($y_{\text{win}} = y_2$, $y_{\text{lose}} = y_1$) for training DPO model, denoted by DPO*. Similarly, we train SFT model with instruction data {<x_1, y_1>, <x_2, y_2>}, denoted by SFT*. We can observe that increasing the token quantity does indeed yield better performance on some datasets. For example, compared to DPO, DPO* has achieved a performance improvement on IFEval (S-Acc, L-Acc) with Qwen2-7B as the base model. However, there are also cases of decline, such as comparing DPO* with DPO on CFBench (CSR, PSR), which indicates that it is not the case that more tokens always lead to better performance.

At the same time, although consuming the same number of tokens, SFT and DPO still have certain gaps compared to the proposed IOPO, which confirms that the performance improvement of our IOPO does not primarily come from using more tokens, but rather from better constraint-aware modeling of input-output preferences.

7 Conclusion

This paper focuses on the ability of LLMs to follow complex instructions, and introduces **TRACE**, a multi-constraint complex instruction benchmark which consists of 120K training samples and 1K test cases. Furthermore, we propose **IOPO** alignment method by taking both input and output preferences into account, enabling LLMs to directly learn response preferences and subtly perceive constraints in instructions. The empirical results from extensive testing across in-domain and out-of-domain datasets demonstrate the efficacy of IOPO, with notable improvements of 2.18% and 3.13% compared to DPO, respectively. For future work, we expect to introduce a more in-depth reasoning process to improve constraint-aware abilities.

Limitations

In TRACE, the evaluation set has undergone strict manual verification but the training set has not performed this process considering the cost. Although the models trained on the training set have achieved the significant improvements, we believe that if we can further improve the quality of the training set, it will lead to better model performance on effectiveness and generalization of following complex instructions.

Acknowledgements

We would like to thank the anonymous reviewers for their insightful comments and constructive suggestions. We sincerely thank members of the ConvAI Team in Tongyi Lab for their valuable feedback and discussions.

References

Anthropic. 2024. Claude 3.5 sonnet model card addendum.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, et al. 2022a. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. 2022b. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*.

Ralph Allan Bradley and Milton E Terry. 1952. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345.

Guanting Dong, Keming Lu, Chengpeng Li, Tingyu Xia, Bowen Yu, Chang Zhou, and Jingren Zhou. 2024. Self-play with execution feedback: Improving instruction-following capabilities of large language models. *arXiv preprint arXiv:2406.13542*.

Jie Gao, Simret Araya Gebreegziabher, Kenny Tsu Wei Choo, Toby Jia-Jun Li, Simon Tangi Perrault, and Thomas W Malone. 2024. A taxonomy for human-llm interaction modes: An initial exploration. In *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–11.

Yingqiang Ge, Wenyue Hua, Kai Mei, jianchao ji, Juntao Tan, Shuyuan Xu, Zelong Li, and Yongfeng Zhang. 2023. Openagi: When llm meets domain experts. In *Advances in Neural Information Processing Systems*, volume 36, pages 5539–5568. Curran Associates, Inc.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Peter Hase and Mohit Bansal. 2022. When can models learn from explanations? a formal framework for understanding the roles of explanation data. In *Proceedings of the First Workshop on Learning with Natural Language Supervision*, pages 29–39. Association for Computational Linguistics.

Qianyu He, Jie Zeng, Qianxi He, Jiaqing Liang, and Yanghua Xiao. 2024a. From complex to simple: Enhancing multi-constraint complex instruction following ability of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 10864–10882, Miami, Florida, USA. Association for Computational Linguistics.

Qianyu He, Jie Zeng, Wenhao Huang, Lina Chen, Jin Xiao, Qianxi He, Xunzhe Zhou, Jiaqing Liang, and Yanghua Xiao. 2024b. Can large language models understand real-world complex instructions? In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18188–18196.

Jiwoo Hong, Noah Lee, and James Thorne. 2024. ORPO: Monolithic preference optimization without

reference model. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 11170–11189. Association for Computational Linguistics.

David R. Hunter. 2004. MM algorithms for generalized Bradley-Terry models. *The Annals of Statistics*, 32(1):384 – 406.

Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. 2024. Openai o1 system card. *arXiv preprint arXiv:2412.16720*.

Yuxin Jiang, Yufei Wang, Xingshan Zeng, Wanjun Zhong, Liangyou Li, Fei Mi, Lifeng Shang, Xin Jiang, Qun Liu, and Wei Wang. 2024. FollowBench: A multi-level fine-grained constraints following benchmark for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4667–4688. Association for Computational Linguistics.

Callie Y Kim, Christine P Lee, and Bilge Mutlu. 2024. Understanding large-language model (llm)-powered human-robot interaction. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, pages 371–380.

Yizhi Li, Ge Zhang, Xingwei Qu, Jiali Li, Zhaoqun Li, Noah Wang, Hao Li, Ruibin Yuan, Yinghao Ma, Kai Zhang, Wangchunshu Zhou, Yiming Liang, Lei Zhang, Lei Ma, Jiajun Zhang, Zuowen Li, Wenhao Huang, Chenghua Lin, and Jie Fu. 2024. CIF-bench: A Chinese instruction-following benchmark for evaluating the generalizability of large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 12431–12446, Bangkok, Thailand and virtual meeting. Association for Computational Linguistics.

Renze Lou, Kai Zhang, and Wenpeng Yin. 2023. A comprehensive survey on instruction following. *arXiv preprint arXiv:2303.10475*.

Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2024. Wizardcoder: Empowering code large language models with evol-instruct. In *The Twelfth International Conference on Learning Representations*.

Yu Meng, Mengzhou Xia, and Danqi Chen. 2024. Simpo: Simple preference optimization with a reference-free reward. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*.

Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. 2022. Cross-task generalization via natural language crowdsourcing instructions. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3470–3487. Association for Computational Linguistics.

Subhabrata Mukherjee, Arindam Mitra, Ganesh Jawahar, Sahaj Agarwal, Hamid Palangi, and Ahmed Awadallah. 2023. Orca: Progressive learning from complex explanation traces of gpt-4. *arXiv preprint arXiv:2306.02707*.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc.

Yiwei Qin, Kaiqiang Song, Yebowen Hu, Wenlin Yao, Sangwoo Cho, Xiaoyang Wang, Xuansheng Wu, Fei Liu, Pengfei Liu, and Dong Yu. 2024. InFoBench: Evaluating instruction following ability in large language models. In *Findings of the Association for Computational Linguistics ACL 2024*, pages 13025–13048. Association for Computational Linguistics.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems*, volume 36, pages 53728–53741. Curran Associates, Inc.

Feifan Song, Bowen Yu, Minghao Li, Haiyang Yu, Fei Huang, Yongbin Li, and Houfeng Wang. 2024. Preference ranking optimization for human alignment. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18990–18998.

Haoran Sun, Lixin Liu, Junjie Li, Fengyu Wang, Baohua Dong, Ran Lin, and Ruohui Huang. 2024. Conifer: Improving complex constrained instruction-following ability of large language models. *arXiv preprint arXiv:2404.02823*.

Minzheng Wang, Longze Chen, Cheng Fu, Shengyi Liao, Xinghua Zhang, Bingli Wu, Haiyang Yu, Nan Xu, Lei Zhang, Run Luo, et al. 2024a. Leave no document behind: Benchmarking long-context llms with extended multi-doc qa. In *EMNLP*. Association for Computational Linguistics.

Minzheng Wang, Xinghua Zhang, Kun Chen, Nan Xu, Haiyang Yu, Fei Huang, Wenji Mao, and Yongbin Li. 2024b. Reframing dialogue interaction with fine-grained element modeling. *arXiv preprint arXiv:2412.04905*.

Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, and Zhifang Sui. 2024c. Large language models are not fair evaluators. In *Proceedings of ACL*, pages 9440–9450. Association for Computational Linguistics.

Bosi Wen, Pei Ke, Xiaotao Gu, Lindong Wu, Hao Huang, Jinfeng Zhou, Wenchuang Li, Binxin Hu,

Wendy Gao, Jiaxin Xu, Yiming Liu, Jie Tang, Hongning Wang, and Minlie Huang. 2024. Benchmarking complex instruction-following with multiple constraints composition. In *Advances in Neural Information Processing Systems*, volume 37, pages 137610–137645. Curran Associates, Inc.

Tingyu Xia, Bowen Yu, Yuan Wu, Yi Chang, and Chang Zhou. 2024. Language models can evaluate themselves via probability discrepancy. In *Findings of ACL*, pages 4889–4901. Association for Computational Linguistics.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. 2024. WizardLM: Empowering large pre-trained language models to follow complex instructions. In *The Twelfth International Conference on Learning Representations*.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, et al. 2024a. Qwen2 technical report. *arXiv preprint arXiv:2407.10671*.

Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024b. Harnessing the power of llms in practice: A survey on chatgpt and beyond. *ACM Transactions on Knowledge Discovery from Data*, 18(6):1–32.

Qinyuan Ye and Xiang Ren. 2021. Learning to generate task-specific adapters from task description. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 2: Short Papers)*, pages 646–653. Association for Computational Linguistics.

Zhiyuan Zeng, Jiatong Yu, Tianyu Gao, Yu Meng, Tanya Goyal, and Danqi Chen. 2024. Evaluating large language models at evaluating instruction following. In *Proceedings of ICLR*.

Tao Zhang, Yanjun Shen, Wenjing Luo, Yan Zhang, Hao Liang, Fan Yang, Mingan Lin, Yujing Qiao, Weipeng Chen, Bin Cui, et al. 2024a. Cfbench: A comprehensive constraints-following benchmark for llms. *arXiv preprint arXiv:2408.01122*.

Wenyuan Zhang, Jiawei Sheng, Shuaiyi Nie, Zefeng Zhang, Xinghua Zhang, Yongquan He, and Tingwen Liu. 2024b. Revealing the challenge of detecting character knowledge errors in llm role-playing. *arXiv preprint arXiv:2409.11726*.

Xinghua Zhang, Bowen Yu, Haiyang Yu, Yangyu Lv, Tingwen Liu, Fei Huang, Hongbo Xu, and Yongbin Li. 2023. Wider and deeper llm networks are fairer llm evaluators. *Preprint*, arXiv:2308.01862.

Xinghua Zhang, Haiyang Yu, Yongbin Li, Minzheng Wang, Longze Chen, and Fei Huang. 2024c. The imperative of conversation analysis in the era of llms: A survey of tasks, techniques, and trends. *arXiv preprint arXiv:2409.14195*.

Yaowei Zheng, Richong Zhang, Junhao Zhang, Yanhan Ye, Zheyan Luo, Zhangchi Feng, and Yongqiang Ma. 2024. Llamafactory: Unified efficient fine-tuning of 100+ language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, Bangkok, Thailand. Association for Computational Linguistics.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*.

A Taxonomy of Constraint

Constraint Type	Constraint Dimension	Description
Content Constraint	Theme Constraint	The generated content should focus on a specific topic or field.
	Exclusion Constraint	Clearly specify the information or content that should not be included in the generated content.
	Inclusion Constraint	Clearly specify the particular information or content that must be included in the generated content.
	Value Constraint	The generated content should not contain information that violates values, such as safety, false information, discrimination, or bias.
	Privacy Constraint	The generated content should not include details that may infringe on privacy, such as personal data or sensitive information.
	Numerical Constraint	Limit the length and number of words, sentences, and paragraphs in the generated content, or use numerical precision constraints to ensure accuracy.
Situation Constraint	Role-Playing Constraint	The generated content should be based on a specific role or situational background.
	Target Audience Constraint	The generated content should target a specific audience, which affects the terminology used, the level of detail provided, and the complexity of the content.
	Prior Condition Constraint	When a specific intention is met, a particular process should be followed to perform an operation or output specific content.
	Natural Language Process Background Information Constraint	Add natural language form process information, such as procedures or business processes, to assist in generating answers.
	Markdown Process Background Information Constraint	Add markdown-formatted process information, such as procedures or business processes, to assist in generating answers.
	Table Background Information Constraint	Background information is presented in table form, providing a series of markdown-formatted tables to assist in generating answers.
	Text Background Information Constraint	Background information is presented in text form, providing a series of textual background information to assist in generating answers.
Style Constraint	Tone and Style Constraint	The generated content should adopt a specific tone and style, such as formal, polite, academic, concise, literary, romantic, or sci-fi.
	Emotion Constraint	The generated content should express a specific emotion or mood, such as ensuring the content is positive, inspiring, or empathetic.
	Linguistic Characteristics Constraint	Use specific linguistic features, such as metaphors, personification, and other rhetorical devices.
	Multilingual Constraint	The content should be generated in a specific language or switch between languages according to complex patterns.
Format Constraint	Output Format Constraint	The generated content should be in a specific data format, such as tables, JSON, HTML, LaTeX, or Markdown.
	Text Pattern Constraint	Use specified fonts and font sizes, or special emoji, to ensure readability across different devices and platforms.
	Grammar Structure Constraint	The generated content should strictly follow specific grammatical structures, such as subject-predicate-object, subject-verb, etc.
	Citation Constraint	The generated content should include citations to sources, providing reliable sources and literature support; follow specific citation formats or reference styles.
	Numbering and List Constraint	The generated content should use numbered lists or bullet points to organize information.
	Hierarchical Structure Constraint	The generated content should be organized according to a specific hierarchical structure, such as using headings and subheadings.
	Template Constraint	The generated content should follow a specific layout or format, such as text alignment, paragraph indentation, and structural templates like introduction-body-conclusion.
Example Constraint	Positive Example Constraint	Provide examples that meet the requirements, and require the model to generate content based on these examples.
	Negative Example Constraint	Provide examples that do not meet the requirements, and require the model to avoid generating content similar to these examples.

Table 6: Five constraint types and 26 constraint dimensions with their corresponding descriptions.

B Prompt

B.1 Constraint Expansion Prompt

[Task Prompt]
You are an instruction enhancer. Given an instruction, you need to modify it by adding constraints to make it more complex. You can choose several appropriate types of constraints from those given below, but you must maintain the thematic consistency of the original instruction.
{Constraints}
[Input]
—INPUT—
<Instruction>:
{Instruction}
—OUTPUT—

B.2 Instruction Structuring Prompt

[Task Prompt]
You are provided with an instruction. As a prompt engineer, your task is to extract the task description, constraints, and the input contained in the given instruction.
[Requirements]
If there is no constraints information that can be extracted from the instruction, only output NULL in the constraints field.
If there is no input information that can be extracted from the instruction, only output NULL in the input field.
Information in the input field and constraints field cannot be duplicated.
Information in the input field and task description field cannot be duplicated.
Information in the task description field and constraints field cannot be duplicated.
The content extracted for the task description, constraints, and input elements should be consistent with the semantics of the instruction to be extracted.
Evaluate the quality of the instruction; if the instruction is poor, incomplete, or contradictory, do not perform constraints extraction.
[Input]
—INPUT—
<Instruction>:
{Instruction}
—OUTPUT—

B.3 Judge Completeness Prompt

[Task Prompt]
You are an instruction integrity discriminator, capable of determining whether a given instruction is complete.
[Requirements]
The given instruction consists of three parts: <Task Description, Constraints, Input>, where Input can be NULL. You can refer to the examples given in Example, but you should not directly copy the examples.
[Example]
{Examples}
[Input]
—INPUT—
{Instruction}
—OUTPUT—

B.4 Judge Redundancy Prompt

[Task Prompt]
You are the redundancy detector for instructions, capable of determining whether given instructions are redundant.
[Requirements]
The given instruction consists of <Task Description, Constraints, Input>, where Input can be NULL; You can refer to the Examples provided, but you should not directly copy the examples.
[Example]
{Examples}
[Input]
—INPUT—
{Instruction}
—OUTPUT—

B.5 Response Evaluation Prompt

[System]
You are a fair judge, and please evaluate the quality of an AI assistant's responses to user query. You need to assess the response based on the following constraints. We will provide you with the user's query, some constraints, and the AI assistant's response that needs your evaluation. When you commence your evaluation, you should follow the following process:
1. Evaluate the AI assistant's response on different constraints, and after each constraint evaluation, assign a score from 0 to 10.
2. Aggregate the assessments from each constraint to give an overall score for the AI assistant's response, ranging from 0 to 10.
3. Your scoring should be as strict as possible, overall, the higher the quality of the model's response, the higher the score.
4. When the model's response is irrelevant to the question, or contains significant factual errors, or generates harmful content, the Constraints Overall Score must be 0 points.
5. It is necessary to strictly follow the format in the [Example] for generation, the Fine Grained Score format is Json, and Constraints Overall Score format is List.
Please remember to provide evaluations and explanations before your scoring. After your explanation of each constraint, include a score for that constraint.
[Example]
{Examples}
[Input]
—INPUT—
#Task Description:
{task_description}
#Constraints:
{constraint}
#Input:
{input}
#Response:
{answer}
—OUTPUT—

C DPO-Series Data Construction

We construct DPO training data based on TRACE training set by prompting Qwen2-72B-Instruct to generate a worse response y_{loose} compared to original response y_{win}. The construction process is

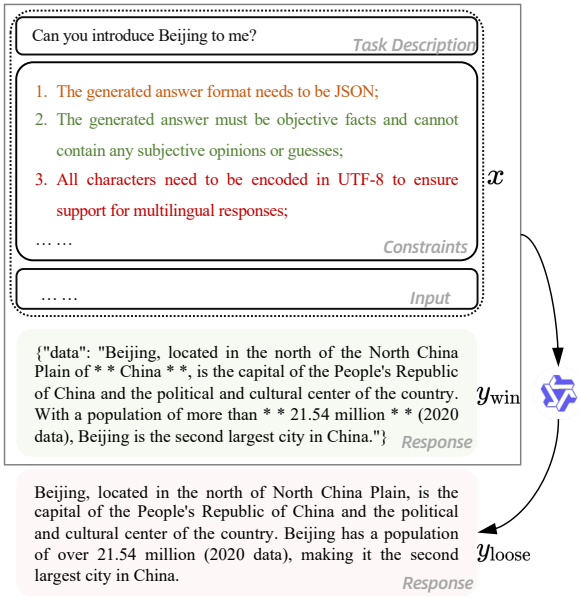

Figure 6: DPO-series Data Construction.

depicted in Figure 6, the prompt is shown as follows:

> #Task Description:
> ***{task_description}***
> #Constraints:
> ***{constraint}***
> #Input:
> ***{input}***
> #Ref:
> The provided answer is: ***{response}***
> According to #Task Description, #Constraints and #Input, please generate a **Worse** answer in terms of complying with the #Constraint than the provided one.
> Please ONLY output the answer.

D IOPO Data Construction

We construct IOPO training data based on TRACE training set by the following steps (the detailed process is shown in Figure 7):

Step 1: prompting Qwen2-72B-Instruct to generate new constraints by "add", "remove", and "revise" operations, making the response not comply with the new constraints, and then the task description, new constraints, and input are combined to form x_2. The corresponding prompt is as follows:

x_2 Generation Prompt:

> #Task Description:
> ***{task_description}***
> #Constraints:
> ***{constraint}***
> #Input:
> ***{input}***
> #Ref:
> The provided answer is: ***{response}***
> According to #Task Description, #Constraints and #Input, please ***{OP}*** items of original CONSTRAINTS to generate the new CONSTRAINTS, making the provided answer NOT comply with the new CONSTRAINTS.
> Please ONLY output the new CONSTRAINTS.

OP can be randomly selected from {"ADD new items into the", "DELETE partial", "REVISE specific"} according to a uniform distribution.

Step 2: For instruction x_2, we prompt Qwen2-72B-Instruct to generate the corresponding response y_2. The prompt is *Response Generation Prompt*.

Step 3: We finally prompt Qwen2-72B-Instruct to evaluate the response y_2, and only keep the full-score ones. The prompt is *Response Evaluation Prompt*.

Finally, we prompt Qwen2-72B-Instruct to check the rationality of the group pairs (<x_1, y_1>, <x_2, y_2>, <x_1, y_2>, <x_2, y_1>).

E IOPO Variant

In fact, when implementing IOPO in the business applications, we can adopt the following simplification by omitting $\Pi_2(\pi_\theta)$ in Eq.9 for less memory usage:

$$\begin{aligned}\mathcal{L}_{\text{IOPO}^*}(\pi_\theta) = -\mathbb{E}_{i\sim D}\Bigg\{\log\Bigg[\sigma\Bigg(2\beta\log\frac{\pi_\theta(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} \\ -\beta\log\frac{\pi_\theta(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} - \beta\log\frac{\pi_\theta(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)}\Bigg)\Bigg]\Bigg\} \\ i = < x_1, y_1, x_2, y_2 >\end{aligned} \tag{11}$$

After simplification, the memory usage is approximately 1.5 times that of DPO, and the performance achieves average increase of 2.03% compared to DPO, decreases by 0.48% compared to before IOPO simplification.

F Compared to Mainstream LLMs

As shown in Table 7, we evaluate some mainstream closed-sourced/SOTA models, including OpenAI o1 (Jaech et al., 2024), DeepSeek-R1 (Guo et al., 2025), and Claude-3.5-Sonnet (Anthropic, 2024), and observe that: (1) Our benchmark TRACE provides the consistent rank (Claude-3.5-Sonnet > o1 > DeepSeek-R1) as existing benchmarks IFEval, CFBench, and COMPLEXBENCH, confirming the data's quality. (2) Larger closed-sourced/SOTA models have better performance than 7B/8B weaker models. However, the proposed algorithm IOPO significantly narrows the gap between closed-sourced/SOTA models and 7B/8B

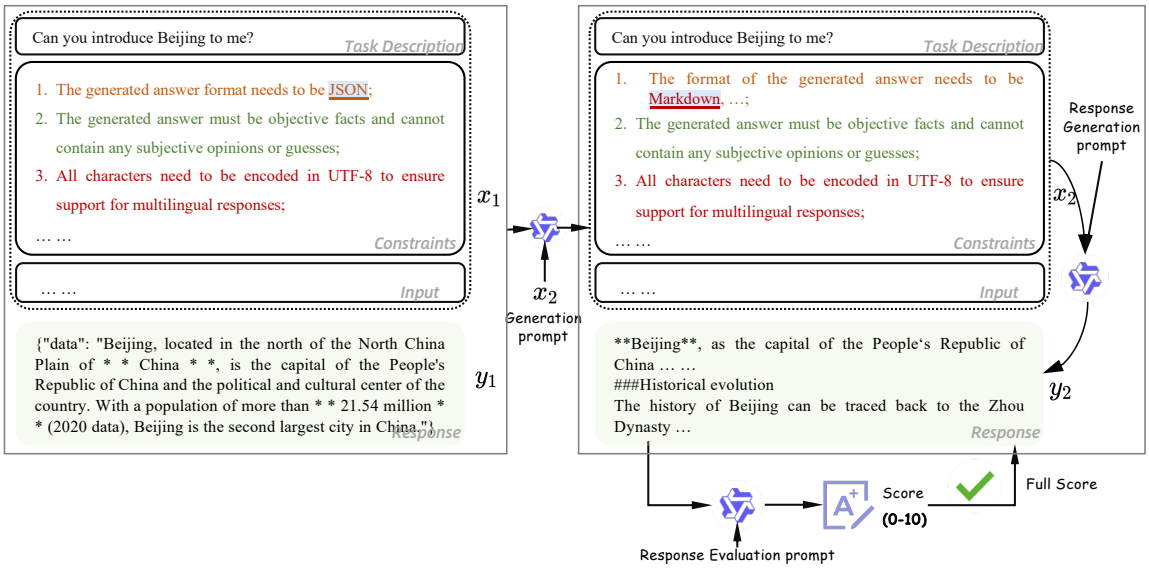

Figure 7: IOPO Data Construction.

Model	Method	TRACE		IFEval		CFBench			COMPLEXBENCH
		IF-S	IF-M	S-Acc	L-Acc	CSR	ISR	PSR	DRFR
Qwen2-7B	Instruct	72.5	54.5	51.6	56.4	75.8	39.1	50.2	68.1
	SFT	76.0	56.1	52.3	54.2	77.8	40.4	52.9	68.2
	PPO	77.0	57.7	51.4	53.8	76.2	38.8	50.6	68.6
	ORPO	77.9	61.7	53.1	56.9	79.7	45.9	57.0	69.1
	SimPO	78.3	63.6	52.2	57.6	78.4	45.0	57.6	67.8
	DPO	79.0	67.2	52.7	58.2	80.0	45.1	57.9	70.9
	IOPO (Ours)$_{Improv.}$	82.0$_{\uparrow 3.0}$	68.9$_{\uparrow 1.7}$	59.9$_{\uparrow 7.2}$	63.6$_{\uparrow 5.4}$	80.7$_{\uparrow 0.7}$	47.0$_{\uparrow 1.9}$	58.7$_{\uparrow 0.8}$	72.6$_{\uparrow 1.7}$
Llama3.1-8B	Instruct	67.5	52.9	74.3	78.6	71.4	35.7	46.9	63.2
	SFT	75.5	62.9	71.0	74.1	78.4	43.2	54.7	68.2
	PPO	75.0	57.3	69.9	72.3	75.9	40.9	50.7	68.1
	ORPO	77.0	63.1	72.3	77.3	79.4	46.6	57.2	69.6
	SimPO	76.3	64.5	71.2	76.6	80.6	47.8	58.7	70.2
	DPO	79.0	69.2	71.5	76.5	80.8	48.1	59.8	70.8
	IOPO (Ours)$_{Improv.}$	81.5$_{\uparrow 2.5}$	70.7$_{\uparrow 1.5}$	78.2$_{\uparrow 6.7}$	81.0$_{\uparrow 4.5}$	81.8$_{\uparrow 1.0}$	49.9$_{\uparrow 1.8}$	61.1$_{\uparrow 1.3}$	71.8$_{\uparrow 1.0}$
Mainstream LLMs	OpenAI o1	85.2	74.7	84.6	89.7	86.7	62.4	72.1	81.3
	DeepSeek-R1 (671B)	84.1	73.8	83.5	89.4	86.5	62.1	71.9	78.7
	Claude-3.5-Sonnet	87.9	76.3	86.7	90.5	87.1	62.6	72.3	82.6

Table 7: Performance on TRACE, IFEval, CFBench, and COMPLEXBENCH.

base weaker models, especially on in-domain TRACE data. This inspires us to construct more diverse high-quality data and more advanced algorithm for enabling weaker models to achieve similar performance to larger-scale models.

G Derivation for $p(\mathcal{G}_1 \succ \mathcal{G}_2)$

As described in Eq. 6 as follows:

$$p(\mathcal{G}_1 \succ \mathcal{G}_2) = \frac{e^{r(x_1,y_1)+r(x_2,y_2)}}{e^{r(x_1,y_1)+r(x_2,y_2)} + e^{r(x_1,y_2)+r(x_2,y_1)}} \tag{12}$$

As described in Eq. 4, the reward function $r(x, y)$ can be represented by the policy model π_r as follows:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x) \tag{13}$$

Combining above equations, we can derive that:

$$\begin{aligned}
p^* &= \frac{\exp\left(\beta \log \frac{\pi^*(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} + \beta \log Z(x_1) + \beta \log \frac{\pi^*(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} + \beta \log Z(x_2)\right)}{\exp\left(\beta \log \frac{\pi^*(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} + \beta \log Z(x_1) + \beta \log \frac{\pi^*(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} + \beta \log Z(x_2)\right) + \exp\left(\beta \log \frac{\pi^*(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} + \beta \log Z(x_1) + \beta \log \frac{\pi^*(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)} + \beta \log Z(x_2)\right)} \\
&= \frac{1}{1 + \exp\left(\beta \log \frac{\pi^*(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} - \beta \log \frac{\pi^*(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} + \beta \log \frac{\pi^*(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)} - \beta \log \frac{\pi^*(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)}\right)} \\
&= \sigma\left(\underbrace{\frac{1}{2}\left(2\beta \log \frac{\pi^*(y_1|x_1)}{\pi_{\text{ref}}(y_1|x_1)} - \beta \log \frac{\pi^*(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)} - \beta \log \frac{\pi^*(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)}\right)}_{<x_1, y_1> \text{ for different } x, y} + \underbrace{2\beta \log \frac{\pi^*(y_2|x_2)}{\pi_{\text{ref}}(y_2|x_2)} - \beta \log \frac{\pi^*(y_1|x_2)}{\pi_{\text{ref}}(y_1|x_2)} - \beta \log \frac{\pi^*(y_2|x_1)}{\pi_{\text{ref}}(y_2|x_1)}}_{<x_2, y_2> \text{ for different } x, y}\right)
\end{aligned} \tag{14}$$

where the notations p^* and π^* are, respectively, equivalent to $p(\cdot)$ and π_θ in the main text.